Campaign Performance Prediction and Budget Optimization System

Harishwaran K, Mageshwaran H, and Keerthivarman M

Department of Artificial Intelligence and Machine Learning, Vels Institute of Science, Technology & Advanced Studies (VISTAS), Chennai, India

Project Guide: Mrs. V. P. Kanchana

Abstract—Digital advertising performance varies across platforms, campaign types, markets, and spending levels. This paper presents a campaign decision-support prototype that combines historical-data preparation, a supervised revenue-model training pipeline, and constrained cross-platform budget allocation. The archived training code constructs a scikit-learn pipeline with one-hot encoding for categorical campaign attributes and a Gradient Boosting Regressor for log-transformed revenue. Its stored evaluation record reports a mean absolute error (MAE) of 17,134.34, root mean squared error (RMSE) of 27,743.90, and R-squared of 0.2198 on a random 80/20 split containing 1,440 training and 360 test records. The archived FastAPI allocation endpoint, however, calls a separate optimizer that maximizes platform-specific logarithmic response curves with SLSQP and market-dependent budget bounds; it does not load the saved Gradient Boosting model. Accordingly, the paper describes these as separate components and identifies their integration as unfinished in the supplied source snapshot. The prototype demonstrates the structure of a constrained planning workflow, while its stored model scores, uncertain metric units, random split, and unverified response-curve calibration limit claims about predictive or business performance.

Index Terms—Digital advertising, campaign revenue prediction, budget allocation, Gradient Boosting, constrained optimization, SLSQP, decision support.

I. INTRODUCTION

Digital advertising gives organizations access to multiple channels, but campaign returns depend on spend, platform, campaign type, industry, market, timing, and audience response. A planner must make two related decisions: estimate the likely outcome of a campaign and decide how to distribute a limited budget. A prediction model alone does not solve the allocation problem, while an optimizer is only as useful as the response function it evaluates.

This project develops a web-based decision-support prototype for campaign performance prediction and budget optimization. The project abstract identifies a Gradient Boosting Regressor for revenue estimation and a constrained optimizer for assigning spend. The supplied source archive contains a data-preparation script, a separate supervised training pipeline, a FastAPI service, and a platform response-curve optimizer. Inspection of these files reveals an implementation boundary: the training code saves a fitted Gradient Boosting pipeline, but the archived API endpoint calls the response-curve optimizer without loading that model [6].

The paper makes three practical contributions. First, it documents the data and model workflow present in the source snapshot. Second, it formulates the optimizer's market-specific budget constraints and objective. Third, it reports the model metrics stored with the archive and distinguishes them from the separate endpoint behavior and from a previously observed deployed-interface demonstration. No causal lift, profit improvement, or production accuracy is claimed.

II. RELATED WORK

Advertising allocation research includes both offline prediction followed by optimization and sequential learning methods that update decisions as campaign outcomes arrive. Han and Gabor describe a contextual-bandit system that predicts campaign payouts, extrapolates to budgets, and uses Thompson sampling to balance exploration and exploitation [1]. Their reported production results belong to that system and are not results of this project.

Nuara et al. formulate online joint bid and daily-budget optimization across pay-per-click campaigns as a combinatorial semi-bandit problem, including data-scarcity considerations [2]. Gigli and Stella study parametric contextual bandits for performance marketing and multi-ad-group campaigns [3]. These works model sequential decisions and uncertainty more broadly than the current prototype, which performs a static constrained allocation for a submitted budget. The present implementation should therefore be positioned as an educational decision-support workflow rather than an online learning or bidding system.

III. SYSTEM DESIGN

A. Processing Architecture

The system has a user interface, an HTTP API, a budget-allocation path, and a separate model-training path. The React/Vite client collects a proposed budget and market, then sends those two values to the /optimize endpoint using Axios. FastAPI validates the request and calls optimize_budget. The optimizer selects the platform set, checks whether the submitted total is feasible, maximizes the sum of platform response curves under per-platform bounds, and returns allocations and predicted revenue values. Separately, the offline training script transforms historical campaign data, trains the Gradient Boosting pipeline, and serializes the model and metrics.

React / Vite UI	*FastAPI /optimize*	*SLSQP optimizer*
Budget + market	*Validate + route*	*Allocate + estimate*

Offline training path: historical CSV → preprocessing → Gradient Boosting pipeline → saved model and metrics. The archived API does not load this model.

Fig. 1. Architecture in the supplied source snapshot; model training and API inference are separate paths.

The two paths are intentionally shown separately because the archived endpoint does not load the saved model artifact. The project's stated design connects prediction and allocation; the code snapshot demonstrates the components but does not establish that end-to-end integration. The supplied deployed interface and the ZIP snapshot may also represent different revisions: the archived client uses a localhost API URL and the archived backend permits local development origins. Deployment configuration should be matched to a source revision before these settings are attributed to the live application.

B. Data Preparation and Features

The preprocessing script reads a raw campaign CSV, inspects data shape, types, missing values, duplicates, platforms, categories, date coverage, and selected numeric summaries, then writes a processed CSV. It derives year, month, day, day of week, day name, and weekend indicator. It also creates planned_budget as an alias of ad_spend. The training script uses eight inputs: platform, campaign_type, industry, country, month, day_of_week, is_weekend, and planned_budget. Revenue is the target.

The feature list omits post-campaign outcomes such as clicks and conversions, which is appropriate for a pre-campaign estimate. However, because planned_budget is created from ad_spend, the training and serving definitions must be aligned: a proposed budget at inference time must be comparable to the historical spend value used during training. The archive does not fully document the dataset's currency or the provenance of every feature; reported error units therefore remain unspecified.

C. Supervised Revenue Model

Categorical inputs are one-hot encoded with unknown categories ignored, while numeric inputs pass through unchanged. These transformations and the estimator are joined in a scikit-learn Pipeline. The target is transformed using log1p before fitting and inverted with expm1 before metric computation. The archived GradientBoostingRegressor uses 200 estimators, a learning rate of 0.05, maximum tree depth 3, and random_state 42 [4]. The train-test split is random, with 20% held out and random_state 42.

D. Constrained Budget Allocation

The archived optimizer assigns one decision variable b_i to each eligible platform i. For a platform-specific response function $R_i(b_i)$, it maximizes the sum of estimated revenues subject to a fixed total budget and platform bounds:

$$\text{maximize } \Sigma_i R_i(b_i) \text{ subject to } \Sigma_i b_i = B,\ l_i \le b_i \le u_i$$

The response function in the runtime module is $R_i(b) = \alpha_i \ln(1 + \beta_i b) + \gamma_i$, with fixed platform parameters. The code passes the negative total response to SciPy's minimize routine because SLSQP minimizes its objective [5]. Before solving, it checks that the total budget lies between the sum of platform minimums and the sum of platform maximums. It also checks the solver's success flag before assembling the response.

Market	Platforms	Feasible total
India	Google Ads, Meta Ads	₹10,000–₹40,000
Global	Google Ads, Meta Ads, TikTok Ads	₹15,000–₹60,000

For India, each selected platform has a ₹5,000 minimum; Google Ads has a ₹25,000 maximum and Meta Ads a ₹15,000 maximum. Global mode adds TikTok Ads with a ₹5,000 to ₹20,000 range. The API schema applies a broader ₹10,000 to ₹60,000 range, after which the optimizer applies the market-specific feasible range.

IV. IMPLEMENTATION

The backend is implemented with FastAPI and Pydantic. The request contains total_budget and market. A successful response includes total_budget, market, an allocation dictionary, per-platform predicted_revenue, and total_predicted_revenue. The route does not accept campaign name, industry, campaign type, duration, or revenue goal. Although these fields appear in the user interface, the archived request path sends only budget and market; therefore they do not condition the returned allocation in that snapshot.

The client performs basic budget-range checks, shows loading and error states, and clears prior results when the market changes. Its what-if handler sends a second optimizer request for an alternative budget and compares the returned total with the baseline. Such a comparison is a model scenario under the same response assumptions, not a causal estimate of what a real campaign would earn.

The runtime optimizer uses response-curve constants in source code, while separate scripts in the archive fit or inspect budget-response curves. The API path does not load those fitted files dynamically. Reproducible deployment therefore requires recording which coefficient set and source revision are active, ensuring that budget and revenue use consistent units, and testing the API against the intended deployment origin.

V. RESULTS AND EVALUATION

The model metrics JSON included in the archive records the values in Table II. The training script indicates that the scores were calculated after reversing the log transformation, so MAE and RMSE are expressed in the target's original dataset units. The metrics file does not specify the currency or dataset revision, and the values were transcribed from the artifact rather than independently recomputed for this paper.

Metric	Stored value
MAE	17,134.3413
RMSE	27,743.9019
R-squared	0.21984
Training records	1,440
Test records	360

The R-squared value indicates limited explained variation under the stored random split. The result should not be described as strong predictive accuracy. Because observations are randomly split rather than partitioned by time or campaign identity, similar campaigns may appear in both sets; the score may not represent future-period generalization. A stronger evaluation should use a time-aware holdout, compare a simple baseline, report per-platform and per-budget errors, and examine residuals and data coverage.

The allocation code enforces budget equality and individual bounds and raises an error for a failed solve. These are implementation safeguards, not empirical evidence that the response curves predict actual revenue. Validation should compare the optimizer with equal-split and historical-allocation baselines and test boundary budgets, infeasible totals, numerical tolerances, and sensitivity to curve parameters.

VI. DISCUSSION AND LIMITATIONS

The main limitation is the gap between the intended prediction-optimization pipeline and the supplied API implementation. The saved Gradient Boosting model is trained and evaluated separately, whereas /optimize uses fixed logarithmic response curves. Until these components are connected and validated, the stored regression metrics cannot be used to justify the API's channel-level revenue recommendations. In addition, the archived frontend endpoint and backend CORS settings are localhost-oriented; this snapshot alone does not establish the configuration used by the deployed website.

Other limitations include an unspecified metric currency, a random evaluation split, the planned_budget/ad_spend proxy, fixed response parameters, and the absence of uncertainty estimates. Platform returns are added independently, so cross-channel interaction, audience overlap, attribution delay, auction effects, and risk are not represented. UI areas that display demo or synthetic information should be distinguished from outputs returned by a live model or optimizer.

Future work should integrate the trained model or a validated response-curve estimator into the API through a versioned prediction interface; document the dataset source, license, currency, and feature timing; use a time-aware evaluation split; quantify optimizer feasibility and sensitivity; and test the deployed API against the source revision being reported. Measured performance should be added only after the evaluation protocol is reproducible.

VII. CONCLUSION

This paper documents a campaign-planning prototype that combines a historical-data training pipeline with a constrained multi-platform allocation service. The archive contains a Gradient Boosting revenue model and a separate SLSQP optimizer using fixed platform response curves. The stored model metrics and API behavior are reported independently, exposing an integration gap that limits end-to-end claims. The prototype provides a useful basis for campaign decision support, but its predictive and business value requires stronger validation, clarified units, and a verified connection between model outputs and budget allocation.

ACKNOWLEDGMENT—The authors thank Mrs. V. P. Kanchana and the Department of Artificial Intelligence and Machine Learning, VISTAS, for guidance and support.

REFERENCES

[1] B. Han and J. Gabor, “Contextual Bandits for Advertising Budget Allocation,” in Proc. AdKDD, 2020. [Online]. Available: https://www.adkdd.org/papers/contextual-bandits-for-advertising-budget-allocation/2020

[2] A. Nuara, F. Trovò, N. Gatti, and M. Restelli, “Online Joint Bid/Daily Budget Optimization of Internet Advertising Campaigns,” Artificial Intelligence, vol. 305, Art. no. 103663, 2022, doi: 10.1016/j.artint.2022.103663.

[3] M. Gigli and F. Stella, “Multi-armed bandits for performance marketing,” International Journal of Data Science and Analytics, vol. 20, pp. 151–165, 2025, doi: 10.1007/s41060-023-00493-7.

[4] scikit-learn developers, “GradientBoostingRegressor,” scikit-learn API Reference. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html

[5] SciPy developers, “minimize(method='SLSQP'),” SciPy Reference Guide. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/optimize.minimize-slsqp.html

[6] Project team, Campaign Performance Prediction and Budget Optimization source archive, supplied project files, 2026.

[7] Campaign Performance Prediction and Budget Optimization System, deployed web application, accessed Oct. 30, 2026. https://campaign-budget-optimizer-cznq.vercel.app/